Governança, segurança e ética no uso de Inteligência Artificial

TaskAnalyzer | Relatório consolidado após homologação técnica | 25 de outubro de 2026

Projeto	TaskAnalyzer – analisador de tarefas em Python 3.11+
Base técnica	Contrato SDD da Fase 1 (versão de 13/09/2026) e requisitos das Fases 2 e 3
Repositório	github.com/Pedrobsz/sdd-task-analyzer
Revisão versionada	Pull Request #1, integrado à branch main; dois checks aprovados

1. Objetivo e método

A entrega final consolida a especificação orientada por contrato (SDD), a implementação do analisador, o Test Harness com pytest, a execução de CI/CD com GitHub Actions e a execução isolada com Docker Compose. A IA apoiou a elaboração e a revisão técnica; sua saída foi confrontada com as regras documentadas e submetida a testes. A aprovação automática não substitui a homologação humana das decisões de negócio.

2. Rastreabilidade com a especificação inicial

Regra contratual da Fase 1	Implementação e verificação
id_tarefa inteiro ≥ 0; prioridades baixa, média, alta	Validação de identificadores e enums; testes de formatos inválidos.
data_criacao, data_inicio e prazo obrigatórios; fuso UTC	Datas ISO 8601 com UTC; validações de obrigatoriedade, ordem temporal e timezone.
Status concluída, pendente e cancelada	Cálculos de duração e atraso restritos a tarefas concluídas.
Duração = data_conclusao − data_inicio, em minutos	Média de conclusão geral e por prioridade, com duas casas decimais.
Atraso se data_conclusao > prazo	Apenas concluídas podem estar em atraso; igualdade não caracteriza atraso.
Saída principal com quatro métricas previstas na Fase 1	analyze_tasks mantém quatro chaves; resumo_fase2 expõe métricas adicionais separadamente.

3. Riscos de alucinação e controles

Falha potencial no código gerado por IA	Controle adotado
Calcular tempo desde a criação, e não desde o início	Caso de teste distingue data_criacao de data_inicio.
Converter minutos em horas na interface original	Métricas em minutos no contrato original; conversão isolada no resumo da Fase 2.

Falha potencial no código gerado por IA	Controle adotado
Desconsiderar cancelada ou remover acentos	Validação explícita dos estados e das prioridades da Fase 1.
Adicionar dependências externas ou persistência	Aplicação utiliza biblioteca padrão; sem banco e sem armazenamento de tarefas.
Aceitar silenciosamente datas impossíveis	Exceção TaskValidationError com mensagem específica; saída de erro na CLI.

4. Divergências tratadas entre as fases

Datas inconsistentes. A regra 1.3.3 da Fase 1 sugere ignorar registros inconsistentes, mas seu Cenário 2 exige exceção tratada e a Fase 2 nomeia TaskValidationError. O projeto prioriza a rejeição explícita de entradas inválidas e evita agregados parciais; a escolha está documentada para eventual revisão acadêmica.

Unidades e formato das saídas. A Fase 1 exige tempo_medio_conclusao_min e quatro chaves principais; a Fase 2 solicita médias em horas e contagens por status. As métricas complementares foram isoladas na interface resumo_fase2 para preservar analyze_tasks.

Denominador da taxa de atraso. A implementação divide o número de concluídas atrasadas pelo total de concluídas; tarefas pendentes e canceladas não participam. Sem concluídas, retorna 0,0%. Trata-se de uma interpretação explicitada, não de uma fórmula literal fornecida na Fase 1.

Quantidade de tarefas. O total considera todas as entradas válidas, inclusive tarefas pendentes e canceladas; as métricas de tempo e atraso usam apenas concluídas.

5. Evidências da execução técnica

Verificação	Resultado observado e fonte de evidência
Pytest local	65 casos aprovados; captura do terminal local (65 passed in 1.73s).
Exemplo do contrato	5 tarefas, 3 concluídas, média geral de 80 min e 33,33% de atraso (1 de 3 concluídas).
Docker Desktop / Compose local	65 casos aprovados em 0,85s; serviço tests terminou com código 0.
GitHub Actions — Pull Request #1	2 checks aprovados: Test Harness Python 3.11 e Docker Compose (ambiente isolado).
Versionamento	PR #1 integrado à main; commit de merge 5b15b1d; git status local limpo e sincronizado com origin/main.
CI após integração	Os checks confirmados nas capturas são do PR; não se afirma uma nova execução pós-merge sem evidência específica.

6. Homologação humana (Critical-in-the-Loop)

O estudante deve revisar o contrato da Fase 1, examinar os cálculos de duração e atraso, verificar as regras das prioridades/status, executar os exemplos e explicar as escolhas adotadas para os pontos ambíguos. Os 65 testes aprovados verificam os cenários implementados, mas **não demonstram 100% de cobertura do código**, nem comprovam a ausência de defeitos fora do escopo testado.

A revisão do PR de documentação foi registrada no fluxo de branches e integração. A confirmação de uma revisão crítica de todas as decisões de negócio depende da validação humana final do estudante.

7. Segurança, privacidade e sustentabilidade

O módulo processa tarefas em memória; não utiliza API externa, banco de dados ou persistência de registros. Os logs evitam armazenar dados identificáveis das tarefas. A dependência de testes está fixada em requirements.txt, enquanto a aplicação usa a biblioteca padrão. A execução isolada reduz diferenças de ambiente, e o pipeline fornece feedback repetível a cada alteração.

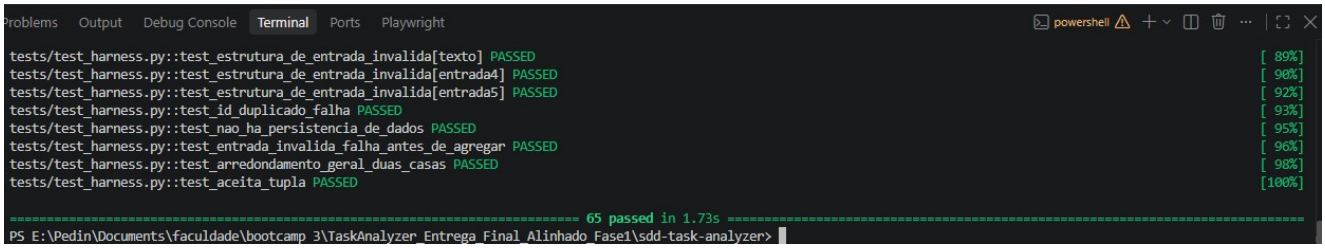
8. Conclusão

As evidências fornecidas demonstram a aprovação da suíte de testes local, a aprovação dentro do Docker Compose e a execução bem-sucedida de ambos os jobs no Pull Request, seguida de merge na main. A implementação está vinculada ao SDD original, com decisões interpretativas explicitadas. A entrega final ainda inclui a apresentação técnica do projeto em vídeo e a submissão dos links exigidos na plataforma AVA.

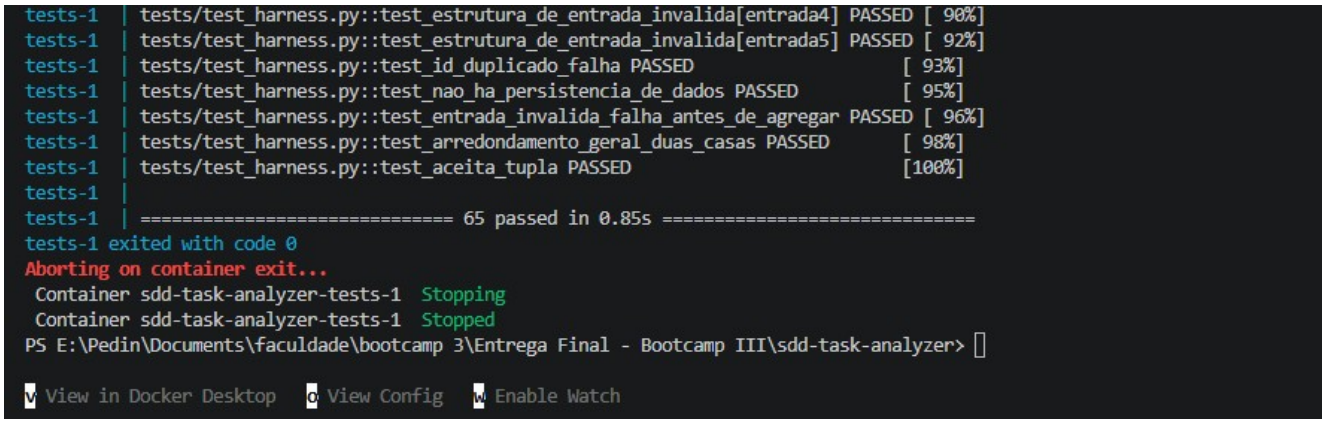
9. Registros visuais da execução

As imagens abaixo são capturas do terminal e do GitHub fornecidas durante a execução do projeto. Não foram geradas artificialmente nem usadas para simular testes.

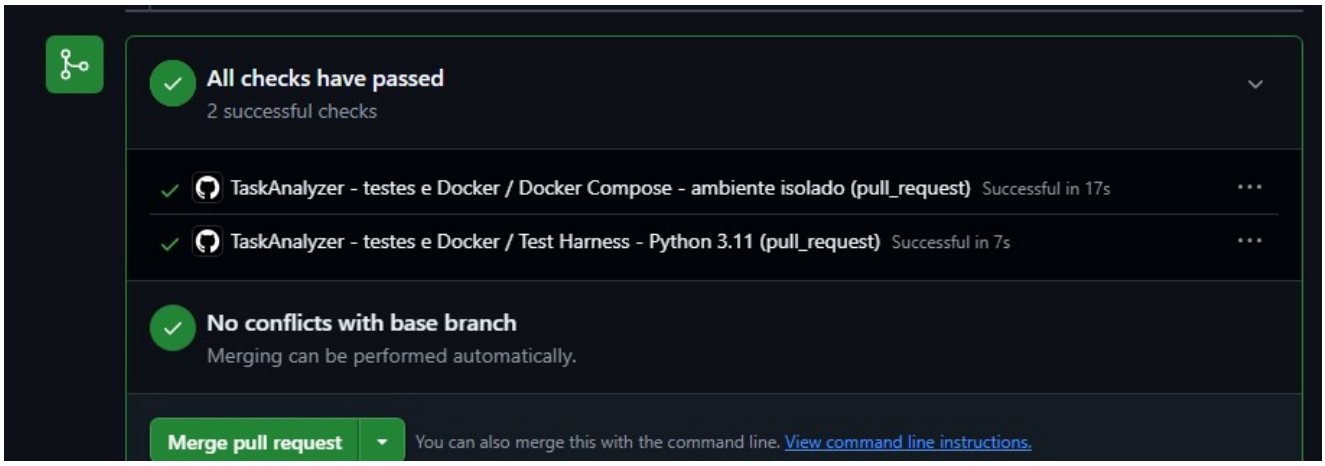
9.1. Pytest executado localmente — 65 casos aprovados



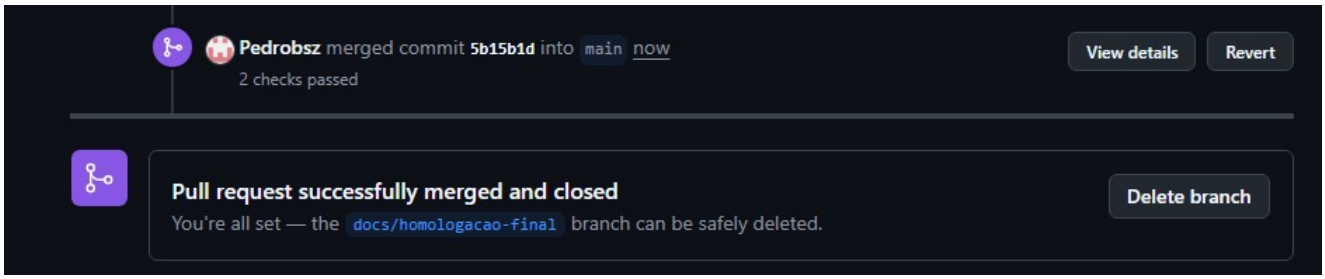
9.2. Docker Compose local — testes aprovados e saída 0



9.3. GitHub Actions no PR — dois checks bem-sucedidos



9.4. PR integrado à main e árvore Git limpa




```
PS E:\Pedin\Documents\faculdade\bootcamp 3\Entrega Final - Bootcamp III\sdd-task-analyzer> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
PS E:\Pedin\Documents\faculdade\bootcamp 3\Entrega Final - Bootcamp III\sdd-task-analyzer>
```

Referências do projeto: repositório público | Pull Request #1